

CprE 487/587 Lab 2 Report (Group 06)

Oct 7, 2026 · @jongwookim · Zach Dixon

1. Overview

In this lab we wrote all the layers of the Lab 1 TinyImageNet CNN in C++ using the course framework. Our outputs match all 12 TensorFlow reference outputs for the three test images. The largest difference for the full model is 4.17×10^{-7} , and we got the same result on the x86 lab machine and on the ZedBoard.

Item	Result
Layers implemented	13 (6 convolution, 3 max pooling, flatten, 2 dense, softmax)
Reference outputs matched	12 of 12, for 3 images, on x86 and ZedBoard
Predicted classes	170, 106, 128 — the same as TensorFlow in Lab 1
Time per image, x86	60.2 ms
Time per image, ZedBoard	2,272 ms
Time per image, TensorFlow on the same x86 machine	34.2 ms (<code>model.predict</code>)
Slowest layer	conv2, 58-60% of the time and 61% of the MACs

Machines: x86 measurements were taken on `co2050-05` (Intel Core i7-12700) with `g++ -O3`, single thread. The ZedBoard (Zynq-7000) ran the same source bare-metal, built with Vitis 2020.1.

Test data: the three images are the ones used in our Lab 1 notebook (pomegranate, miniskirt, German shepherd). The images, the weights and the per-layer TensorFlow outputs were exported in Lab 1 and converted to the framework's file layout by `scripts/import_lab1_data.py`.

2. Layer operations (3.2)

The model has five kinds of layers. ReLU is not a separate layer: it is the activation inside the convolution layers and the first dense layer.

Layer type	Count	Where
Convolution (with ReLU)	6	conv1 to conv6
Max pooling	3	after every two convolutions
Flatten	1	after the last pooling layer
Fully connected	2	dense1 (with ReLU), dense2
Softmax	1	output

2.1 Formulas in tensor index notation

The notation follows the textbook (equation 2.3 and Table 2.1): I is the input feature map, O the output feature map, F the filter weights, b the bias, and U the stride.

Convolution with ReLU. Every convolution in our model has stride $U = 1$ and no padding.

$$O_{\{nmpq\}} = \mathrm{ReLU}\left(\sum_{\{c,r,s\}} I_{\{n\backslash,c\},(Up+r)\backslash,(Uq+s)\backslash}, F_{\{mc\}rs} + b_m\right), \quad P = \frac{H-R+U}{U}, \quad Q = \frac{W-S+U}{U}$$

Fully connected. This is the convolution with $R = H$ and $S = W$, so $P = Q = 1$. With a flattened input it reduces to the form below. dense1 applies ReLU; dense2 has no activation because its output goes to the softmax layer.

$$O_{\{nm\}} = \mathrm{ReLU}\left(\sum_{\{c\}} I_{\{nc\}}\backslash, F_{\{mc\}} + b_m\right) \backslash \text{dense1}, \quad O_{\{nm\}} = \sum_{\{c\}} I_{\{nc\}}\backslash, F_{\{mc\}} + b_m \backslash \text{dense2}$$

Max pooling. There are no weights and each channel is processed on its own. In our model $R = S = 2$ and $U = 2$.

$$O_{\{ncpq\}} = \max_{\{0 \leq r < R, 0 \leq s < S\}} I_{\{n\backslash,c\},(Up+r)\backslash,(Uq+s)\backslash}$$

Flatten. No arithmetic; the values are only re-indexed.

$$O_{\{nk\}} = I_{\{nchw\}}, \quad k = (hW + w)\backslash, C + c$$

Softmax.

$$O_{\{nm\}} = \frac{e^{I_{\{nm\}}}}{\sum_{l=0}^{M-1} e^{I_{\{nl\}}}}$$

ReLU.

$$\mathrm{ReLU}(g) = \max(0, g)$$

2.2 Shape parameters of each layer

$N = 1$ for every layer, because the framework processes one image at a time.

Layer	M	C	H / W	R / S	P / Q	U
conv1	32	3	64	5	60	1
conv2	32	32	60	5	56	1
max pool 1	32	32	56	2	28	2
conv3	64	32	28	3	26	1
conv4	64	64	26	3	24	1
max pool 2	64	64	24	2	12	2
conv5	64	64	12	3	10	1
conv6	128	64	10	3	8	1
max pool 3	128	128	8	2	4	2
dense1	256	2,048	1	1	1	1
dense2	200	256	1	1	1	1

Every row satisfies $P = (H - R + U) / U$. dense1 is listed after flattening ($C = 2,048$, $H = W = 1$); in terms of the unflattened input it is $C = 128$ with $H = W = R = S = 4$.

2.3 Variables and how they appear in our model

Variable	Meaning and purpose	In our code
N	Batch size	Always 1; there is no batch dimension
M	Number of filters, equal to the number of output channels	<code>outDims[2]</code> for convolution, <code>outDims[0]</code> for dense
C	Number of input channels, equal to the channels of each filter	<code>inDims[2]</code>
H, W	Height and width of the input feature map	<code>inDims[0]</code> , <code>inDims[1]</code>
R, S	Height and width of the filter (or pooling window)	<code>weightDims[0]</code> , <code>weightDims[1]</code>
P, Q	Height and width of the output feature map	<code>outDims[0]</code> , <code>outDims[1]</code>
U	Stride: how far the window moves per output	1 for convolution; computed as $H / P = 2$ for pooling

Variable	Meaning and purpose	In our code
I	Input feature map	<code>dataIn</code> , a flat float32 array
F	Filter weights	<code>weightData</code> , a flat float32 array
b	One bias per output channel	<code>biasData</code> , M values
O	Output feature map	<code>getOutputData()</code>

The textbook writes the channel index first, while our data keeps the layout exported from Keras, with the channel last. The formulas mean the same thing; only the position of a value in memory differs.

Tensor	Stored order	Flat index
I, O	(height, width, channel)	$(h * W + w) * C + c$
F, convolution	(R, S, C, M)	$((r * S + s) * C + c) * M + m$
F, dense	(C, M)	$c * M + m$

3. Implementation (3.2)

Each layer is a class in `src/layers/` with a `computeNaive` function written as plain loops, with no optimization. Dimensions are read from the layer parameters, so one convolution class serves all six convolution layers.

Layer	File	How it is computed
Convolution	<code>Convolutional.cpp</code>	Six nested loops: output row, column and filter (p, q, m), then kernel row, column and input channel (r, s, c). The sum starts from the bias and ReLU is applied before the value is stored
Max pooling	<code>MaxPooling.h</code>	For each output position and channel, the maximum over a window whose size is input size / output size (2×2)
Flatten	<code>Flatten.h</code>	One <code>memcpy</code> . The feature map is already stored in (height, width, channel) order, the order Keras flattens in

Layer	File	How it is computed
Dense	<code>Dense.h</code>	For each output, the bias plus the weighted sum of all inputs. A constructor flag turns ReLU off for dense2
Softmax	<code>Softmax.h</code>	Exponential of each input divided by the sum of exponentials. The largest input is subtracted first, which leaves the result unchanged and prevents overflow

The model in `src/ML.cpp` has 13 layers. TensorFlow's last layer applies softmax inside the dense layer; we split it into dense2 and softmax, as the template's comments suggest. Layers 0 to 10 therefore correspond one to one to the 12 TensorFlow layer outputs 0 to 10, and layers 11 and 12 together correspond to TensorFlow output 11.

The framework requires `computeThreaded`, `computeTiled` and `computeSIMD` for every layer. Only the naive version is required for this lab, so in every layer class these call `computeNaive`.

Changes to the framework

We did not change `Layer.h`, `Model.cpp` or any other framework file. The changes outside the layer operations are:

Change	Why
<code>runLayerTest</code> in <code>ML.cpp</code> feeds each layer the TensorFlow output of the previous layer and compares with that layer's own reference, for all three images	The template test always used <code>image_0.bin</code> and <code>layer_0_output.bin</code> , so it could only test layer 0. Testing each layer alone keeps an error in one layer from affecting the others
New <code>runOutputStageTest</code> runs dense2 and softmax together	TensorFlow has no separate output for dense2
<code>runInferenceTest</code> compares with the final reference output and prints the predicted class	The template compared the final output with <code>layer_0_output.bin</code>
New file <code>Flatten.h</code>	The framework checks that a layer's input dimensions equal the previous layer's output dimensions, so (4, 4, 128) to (2048) needs a layer. It uses <code>LayerType::NONE</code> because the framework has no flatten type

Change	Why
<code>scripts/import_lab1_data.py</code> , <code>scripts/plot_layer_times.py</code> , <code>scripts/tf_inference_time.py</code>	Convert the Lab 1 export to the framework's layout, draw the layer-time chart, and time TensorFlow on the same machine

These changes do not affect performance; they only affect what is tested.

4. Verification (3.2, 3.3)

All of our layer outputs match TensorFlow. The largest element-wise difference we saw was 5.72×10^{-6} for a single layer and 4.17×10^{-7} for the full model, which is what we expect from float32 rounding.

We used the framework's `compareWithinPrint`, which reports two values: the maximum element-wise difference (`maxDiff`) and a length-weighted cosine similarity (`compare`). Each layer was tested alone, with the TensorFlow output of the previous layer as its input, for all three images. The errors below are the largest over the three images and are identical on x86 and the ZedBoard.

#	Layer	Maximum error	Cosine similarity
0	conv1	3.94×10^{-7}	100%
1	conv2	7.75×10^{-7}	100%
2	max pool 1	0	100%
3	conv3	4.17×10^{-7}	100%
4	conv4	7.15×10^{-7}	100%
5	max pool 2	0	100%
6	conv5	9.54×10^{-7}	100%
7	conv6	9.54×10^{-7}	100%
8	max pool 3	0	100%
9	flatten	0	100%
10	dense1	5.72×10^{-6}	100%
11 + 12	dense2 + softmax	8.94×10^{-8}	100%

Full model, from the input image to the class probabilities:

Image	Maximum error	Cosine similarity	Predicted class (confidence)	TensorFlow in Lab 1
0 pomegranate	4.17×10^{-7}	99.9999%	170 (0.729327)	170 (0.7293)
1 miniskirt	1.71×10^{-7}	99.9999%	106 (0.103273)	106 (0.1033)
2 German shepherd	2.09×10^{-7}	99.9999%	128 (0.21062)	128 (0.2106)

All 36 layer tests and 3 inference tests pass on both platforms (logs: `results/x86_co2050_run.txt`, `results/zedboard_raw.txt`).

Error tolerance and where the error comes from

Final maximum error. 4.17×10^{-7} for the full model and 5.72×10^{-6} for the worst single layer (dense1).

Tolerance and how we arrived at it. A float32 number only has about 7 significant digits (the relative precision is $2^{-23} \approx 1.2 \times 10^{-7}$). So even a correct implementation can differ from TensorFlow by a few times 10^{-7} relative to the size of the values. Our layer outputs go up to about 2.4 in conv2, 3.5 in conv6 and 11.9 in dense1, so we decided that absolute differences up to about 10^{-5} are acceptable. dense1 has the largest error because its values are the largest and each output is a sum of 2,048 terms. Compared to its largest output, the error is about 5×10^{-7} , which is the same level as the other layers. While debugging we also noticed that a real bug, like a wrong index or a missing bias, gives errors of 10^{-2} or more, so it is easy to tell the two cases apart.

We did not use the framework's pass criterion (cosine similarity above 0.8) as our tolerance because it is too loose. In the basic test, changing one input value by 0.1 still gave 99.996% similarity. That is why we looked at the maximum error instead.

Source of the error, and should there be any? The error comes from the order of the additions. Floating-point addition is not associative, so adding the same numbers in a different order can change the last digits of the result. Our loops add the terms one by one in a fixed order, but TensorFlow uses vectorized operations that add them in a different order. The math is the same, so the only difference is rounding. This also explains why max pooling and flatten have exactly zero error: they only pick or copy values and do no arithmetic. So we think a small error is expected for convolution, dense and softmax, as long as it stays at the rounding level, which it does.

5. Memory and MAC estimate (3.3)

We estimate that the model needs about 4.5 MB of memory and 131.6 million multiply-accumulate operations (MACs) for one image. Most of the memory is in dense1 and most of the computation is in conv2.

All values are 32-bit floats (4 bytes). A layer needs its input, its output and its parameters while it runs.

#	Layer	Input (bytes)	Output (bytes)	Weights + bias (bytes)	Layer total (bytes)	MACs	MAC share
0	conv1	49,152	460,800	9,728	519,680	8,640,000	6.57%
1	conv2	460,800	401,408	102,528	964,736	80,281,600	61.01%
2	max pool 1	401,408	100,352	0	501,760	0	
3	conv3	100,352	173,056	73,984	347,392	12,460,032	9.47%
4	conv4	173,056	147,456	147,712	468,224	21,233,664	16.14%
5	max pool 2	147,456	36,864	0	184,320	0	
6	conv5	36,864	25,600	147,712	210,176	3,686,400	2.80%
7	conv6	25,600	32,768	295,424	353,792	4,718,592	3.59%
8	max pool 3	32,768	8,192	0	40,960	0	
9	flatten	8,192	8,192	0	16,384	0	
10	dense1	8,192	1,024	2,098,176	2,107,392	524,288	0.40%
11	dense2	1,024	800	205,600	207,424	51,200	0.04%
12	softmax	800	800	0	1,600	0	

How the numbers are calculated:

- Feature map bytes = height × width × channels × 4.
- Convolution parameters = $(R \times S \times C \times M + M) \times 4$ bytes; dense parameters = $(C \times M + M) \times 4$ bytes.
- Convolution MACs = $P \times Q \times M \times R \times S \times C$, one per term of the sum for every output value. Dense MACs = $C \times M$.
- Max pooling, flatten and softmax perform no multiply-accumulates. Max pooling does 4 comparisons per output, and softmax does 200 exponentials and 200 divisions.

The "layer total" column cannot be summed, because the output of one layer is the input of the next. For the whole model:

Item	Size
All parameters (770,216 values)	3,080,864 bytes
Output buffers of all 13 layers (349,328 values)	1,397,312 bytes
Input image	49,152 bytes
Total	4,527,328 bytes

The framework allocates every layer's output buffer when the model is built, so this total is what the program holds at once. The parameter and output sizes equal the sizes of the exported weight files and reference output files.

We noticed two things from this table. First, the layer that uses the most memory is not the layer that does the most work: dense1 holds 68% of the parameters but only does 0.4% of the MACs, while conv2 does 61% of the MACs with only 3.3% of the parameters. Second, the MAC share turned out to be a good prediction of the time share we measured, which we show in Section 6.

6. Profiling (3.4, 3.5)

One inference takes 60.2 ms on the x86 lab machine and 2,272 ms on the ZedBoard. TensorFlow takes 34.2 ms on the same x86 machine. On both platforms conv2 uses about 60% of the time.

6.1 Time for a single inference

Platform	Implementation	Time per image	How measured
x86, co2050-05, Intel Core i7-12700	Our C++ (g++ -O3, one thread)	60.2 ms (58.5–61.5)	Framework Timer, 15 measurements
ZedBoard, Zynq-7000	Our C++ (bare metal)	2,272 ms (2,270–2,275)	Framework Timer, 3 images
x86, co2050-05	TensorFlow 2.16.2, one <code>model.predict()</code> call	34.2 ms	Median of 20 runs after a warm-up
x86, co2050-05	TensorFlow, direct <code>model(x)</code> call	7.0–7.3 ms	Median of 20 runs after a warm-up

The timer covers only the 13 layers; loading files and comparing outputs are outside it. As a cross-check, `time ./build/ml` reports 0.403 s real for the whole test program, which agrees with three sets of layer tests (3×62.5 ms) plus three full inferences (3×60.2 ms).

Why the times differ.

- *ZedBoard vs. x86 (38 times slower)*. Both run the same source code, so we think the difference is the processor. The ZedBoard has a small embedded ARM processor, and the lab machine has a recent desktop Intel Core i7-12700. The slowdown is almost the same for every convolution layer (33 to 38 times), so it does not look like one layer is the problem. It looks like the whole board is just slower.
- *Our C++ vs. TensorFlow (1.8 times slower than `predict`, 8.4 times slower than a direct call)*. Our convolution is six nested loops that do one multiplication at a time. TensorFlow runs each convolution as one optimized operation (it showed up as `_MklNativeFusedConv2D` in our Lab 1 profile), which uses SIMD instructions and is written to use the cache well. We also saw in Lab 1 that most of the time of `predict()` is overhead of the call itself, which is why the direct call is so much faster than `predict()`.

6.2 Layer-wise execution time

Layer	x86 (ms)	x86 share	ZedBoard (ms)	ZedBoard share	MAC share
conv1	5.508	8.8%	209.3	9.2%	6.57%
conv2	36.068	57.7%	1,355.7	59.7%	61.01%
max pool 1	0.072	0.1%	2.0	0.1%	
conv3	5.908	9.5%	204.0	9.0%	9.47%
conv4	10.040	16.1%	338.0	14.9%	16.14%
max pool 2	0.026	0.0%	0	0.0%	
conv5	1.746	2.8%	58.0	2.6%	2.80%
conv6	2.268	3.6%	75.0	3.3%	3.59%
max pool 3	0.004	0.0%	0	0.0%	
flatten	0.000	0.0%	0	0.0%	
dense1	0.831	1.3%	27.0	1.2%	0.40%
dense2 + softmax	0.024	0.0%	0	0.0%	0.04%
Sum	62.496		2,269.0		

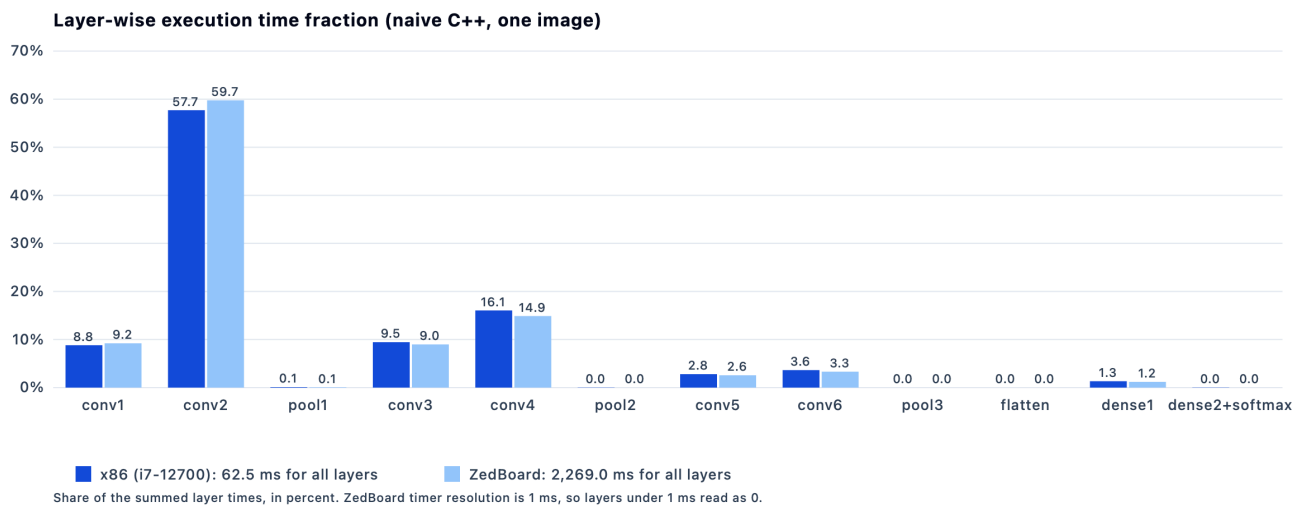


Figure 1. Layer-wise execution time fraction of the naive C++ implementation on the x86 lab machine (Intel Core i7-12700) and the ZedBoard. Each bar is a layer's share of the summed layer times, averaged over the three test images. conv2 takes 57.7% on x86 and 59.7% on the ZedBoard.

Each time is the mean over the three images from the framework `Timer` around one layer. The ZedBoard timer has a resolution of 1 ms, so layers faster than that read as 0.

perf on x86. `perf record --count=100000 ./build/ml` collected 17,513 samples. `perf` attributes time to functions, and all six convolutions share one function, so it confirms the totals but cannot separate the convolution layers; the per-layer values above come from the timers.

Function	Share of cycles
<code>ConvolutionalLayer::computeNaive</code>	97.68%
<code>DenseLayer::computeNaive</code>	1.27%
<code>LayerData::compare</code> and <code>maxDiff</code> (test code)	0.77%
<code>MaxPoolingLayer::computeNaive</code>	0.15%
Everything else	0.13%

The timers give the six convolutions 98.5% of the layer time and dense1 1.3%, in agreement with `perf`.

6.3 Which layers take the time, and why

conv2 takes most of the computation time: 57.7% on x86 and 59.7% on the ZedBoard. All six convolutions together take about 98%.

Reason. Our code is not optimized, so the time of a layer mostly depends on how many MACs it has to do. conv2 uses 5×5 filters with 32 input channels, which is 800 MACs for every output value, and its output is $56 \times 56 \times 32$. That gives 80.3 million MACs, or 61% of the whole model. For every convolution the measured time share is within about three percentage points of its MAC share (conv2: 57.7% and 59.7% measured, 61.0% by MACs). The dense layers have most of the weights but only a few MACs, so they are fast. Pooling, flatten and softmax have no MACs at all.

Does this match the Lab 1 TensorBoard results? Yes. In Lab 1 the same layer (called conv2d_1 in Keras) was also the slowest one. It took 30% of the model's operation time for online inference and 53% at batch size 200, and all the convolutions together took 76% and 94%. The order of the layers is the same as what we see here. Our numbers are closer to the batched TensorFlow profile. We think this is because in the batched case the overhead per operation is small, so the time depends on the MACs, just like in our loops.

Layers ripe for optimization. The convolution layers, starting with conv2 and then conv4, conv3 and conv1. They take about 98% of the time, so speeding up any other layer would barely change the total. They are also a good target because they repeat the same multiply-accumulate 131 million times in a very regular pattern. This kind of loop can be improved with a better loop order or tiling for the cache, SIMD, or multiple threads, and it is also what a hardware MAC unit is designed for, which is where the next labs are going.

Appendix: logs and limits of the measurements

Evidence	File in the repository
x86 test output, all layers and full inference	results/x86_co2050_run.txt
ZedBoard test output	results/zedboard_raw.txt
perf report	results/x86_perf_report.txt
TensorFlow timing on the same machine	results/x86_co2050_tensorflow.txt
Layer-time chart and the script that draws it	assets/layer_time_share.svg, scripts/plot_layer_times.py

Limits of the measurements:

- Three test images were used for verification.
- dense2 has no TensorFlow reference of its own and is verified together with softmax.

- The ZedBoard timer resolves 1 ms, so the four fastest layers have no usable time on the board.
- The TensorFlow time of the layer arithmetic alone was not measured on co2050-05 ; only `predict()` and a direct call were. The Lab 1 profiler figure (about 1.8 ms per image) was taken on a different machine and is not used in the comparison table.
- The optional "Above and Beyond" items were not attempted.